

Quản Lý Độ Phức Tạp Trong Quá Trình Thiết Kế Hệ Thống Tính Toán

Phung Quang Minh
Thread: phgquangminh

Nguồn Gốc Vấn Đề & Động Lực

1. Bạn có thể thấy các hệ thống tính toán hiện đại đều được xây dựng từ hàng tỷ transistor, một ví dụ cụ thể là SoC.
2. SoC là một dạng hệ thống tính toán hoàn chỉnh được thu nhỏ để tích hợp trên một con chip duy nhất.
3. Không một con người nào đủ khả năng hiểu các hệ thống này bằng việc viết và giải phương trình chuyển động của các electron trong từng transistor.

*Việc xây dựng và hiểu hệ thống **đòi hỏi một cách tiếp cận có cấu trúc** nhằm quản lý độ phức tạp.*

Giải Pháp: Trừu Tượng Hóa

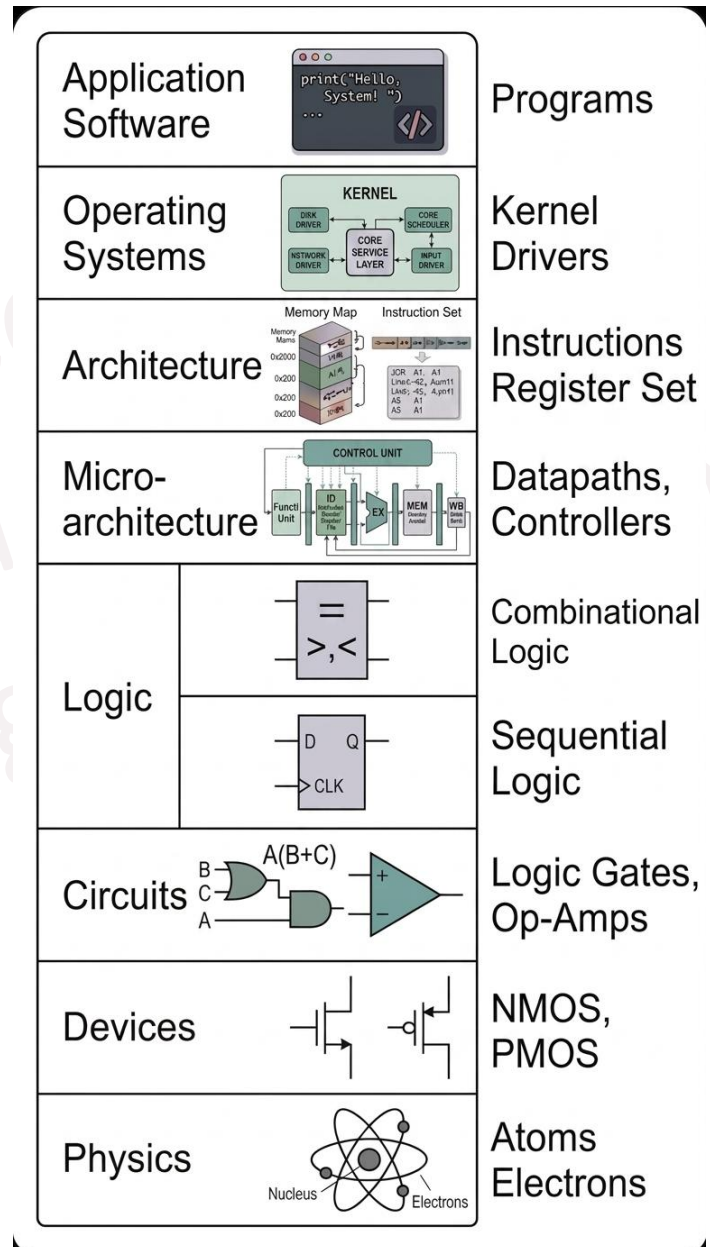
1. Độ phức tạp của hệ thống không thể bị loại bỏ. Thay vào đó, chúng ta tổ chức hệ thống thành các mức mô tả khác nhau.
2. Mỗi mức chỉ tập trung vào những chi tiết cần thiết cho mục đích đang quan tâm.

Các Tầng Trừu Tượng Trong Hệ Thống Tính Toán

1. **Trừu tượng hóa** được thực hiện bằng cách **tổ chức hệ thống thành nhiều tầng khác nhau**.

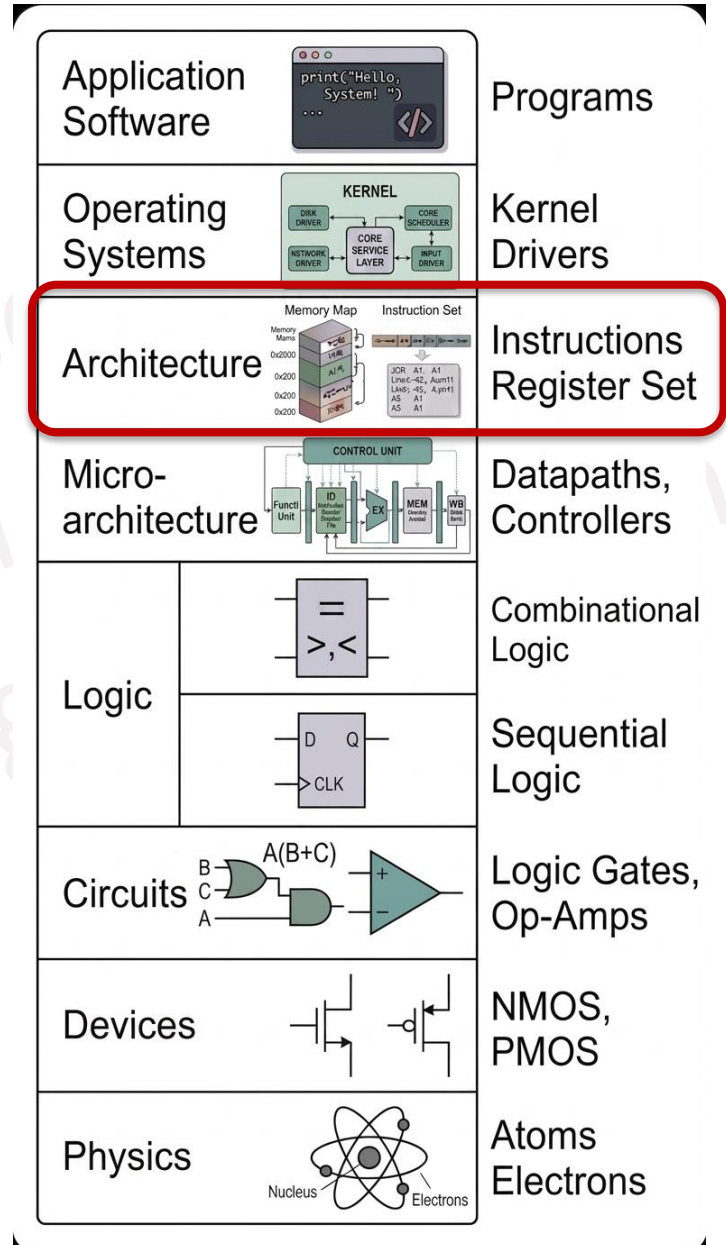
2. Mỗi tầng trừu tượng xác định phạm vi chi tiết cần quan tâm, che giấu chi tiết triển khai của tầng bên dưới và cung cấp một mô hình khái quát hơn cho tầng phía trên.

Cấu trúc phân tầng giúp quá trình thiết kế hệ thống trở nên rõ ràng và có tổ chức cũng như tận dụng lại những dữ liệu đã có.



Tầng Architecture (1/2)

1. Ở tầng **Architecture**, hệ thống được **mô tả từ góc nhìn của phần mềm ở mức mã máy (machine language)**.
2. **Architecture** định nghĩa chính xác **giao diện giữa phần mềm thuần túy và phần cứng**, quy định cách một chương trình bất kỳ điều khiển và quan sát hoạt động của hệ thống.



Tầng Architecture (2/2)

Một số đặc điểm quan tâm chính ở tầng này gồm:

1. Instructions: hệ thống cung cấp những tập lệnh nào để thực hiện việc điều khiển?

2. Register Set:

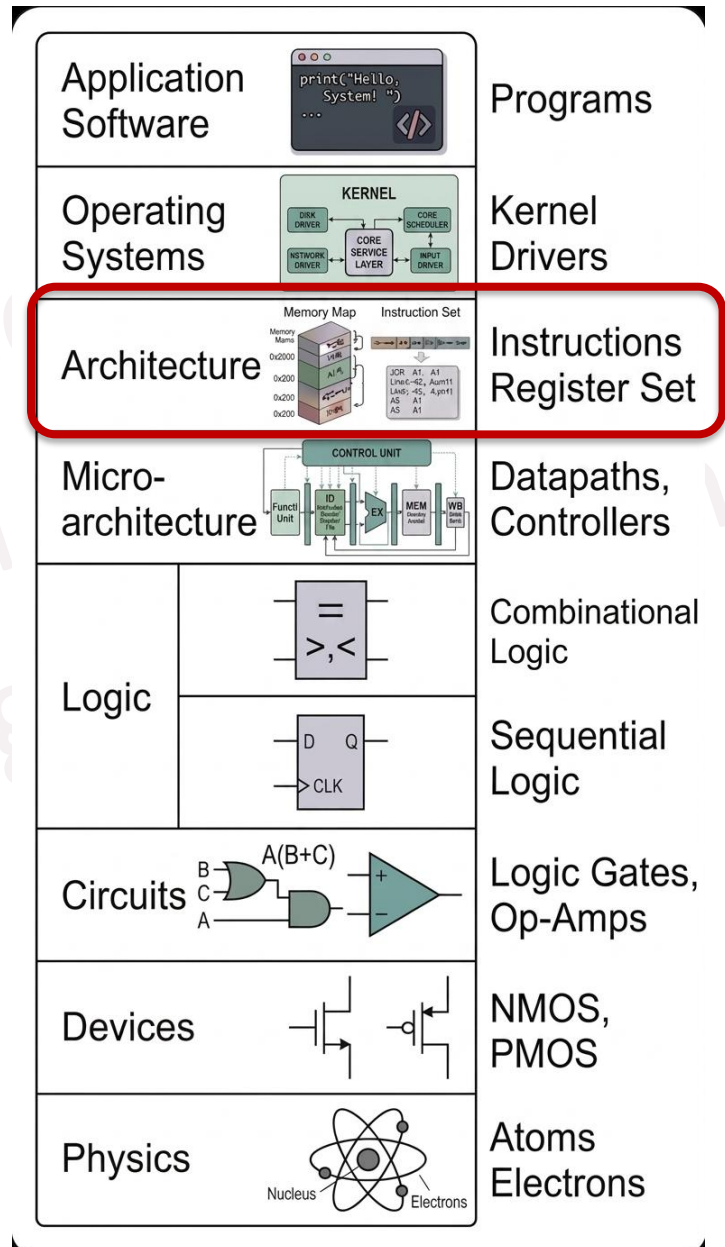
- Kích thước của mỗi thanh ghi là bao nhiêu?
- Có tổng cộng bao nhiêu thanh ghi trong hệ thống để ghi nhớ tạm và theo dõi trạng thái của dữ liệu?

3. Data Types:

- Hệ thống hỗ trợ những kiểu dữ liệu nào?
- Định dạng biểu diễn dữ liệu của hệ thống là Big-Endian hay Little-Endian?

4. I/O Mechanisms:

- Hệ thống trao đổi dữ liệu với các thiết bị ngoại vi thông qua những cơ chế I/O nào?
- Các cổng I/O sẽ được điều khiển và truy cập ra sao?



Tầng Operating Systems

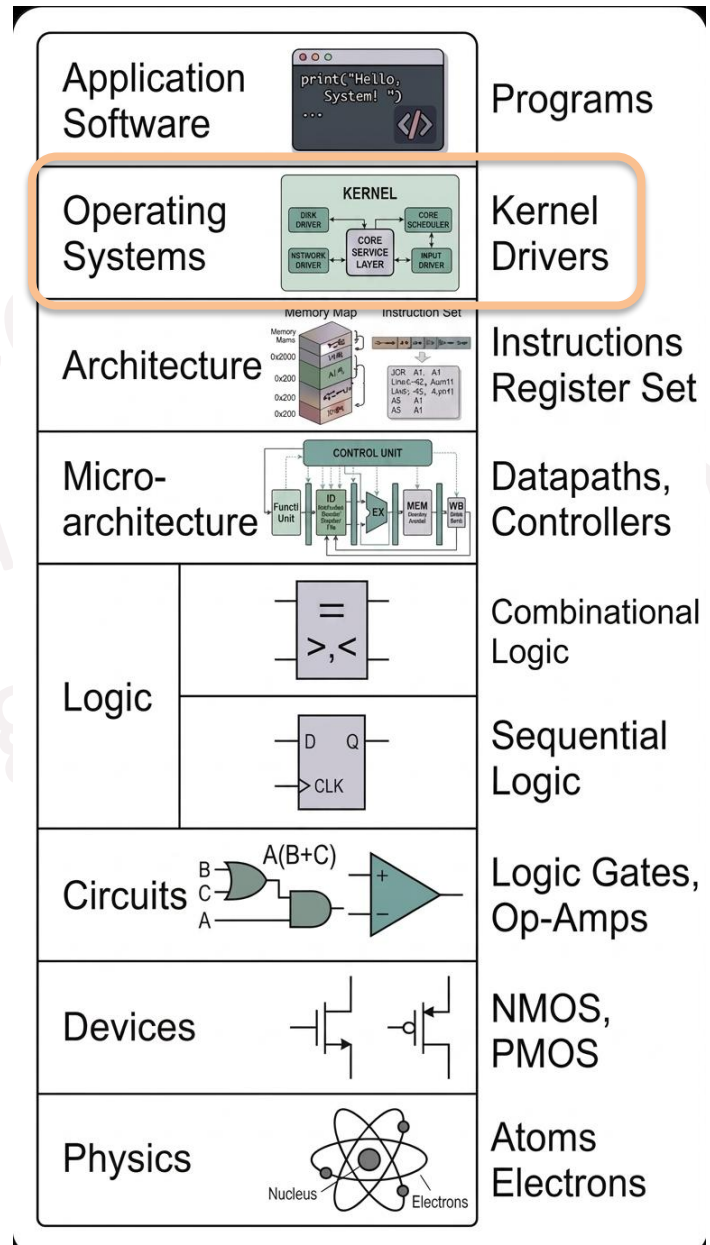
1. Là **phần mềm** hoạt động dựa trên chính Architecture, nhưng đã **che giấu sự phức tạp của cấu trúc mã máy**.
2. Chịu trách nhiệm **quản lý và phân chia các tài nguyên** giới hạn của hệ thống **thông qua 2 cơ chế**:

- **Time Multiplexing:**

Nhiều chương trình luân phiên sử dụng cùng một tài nguyên.

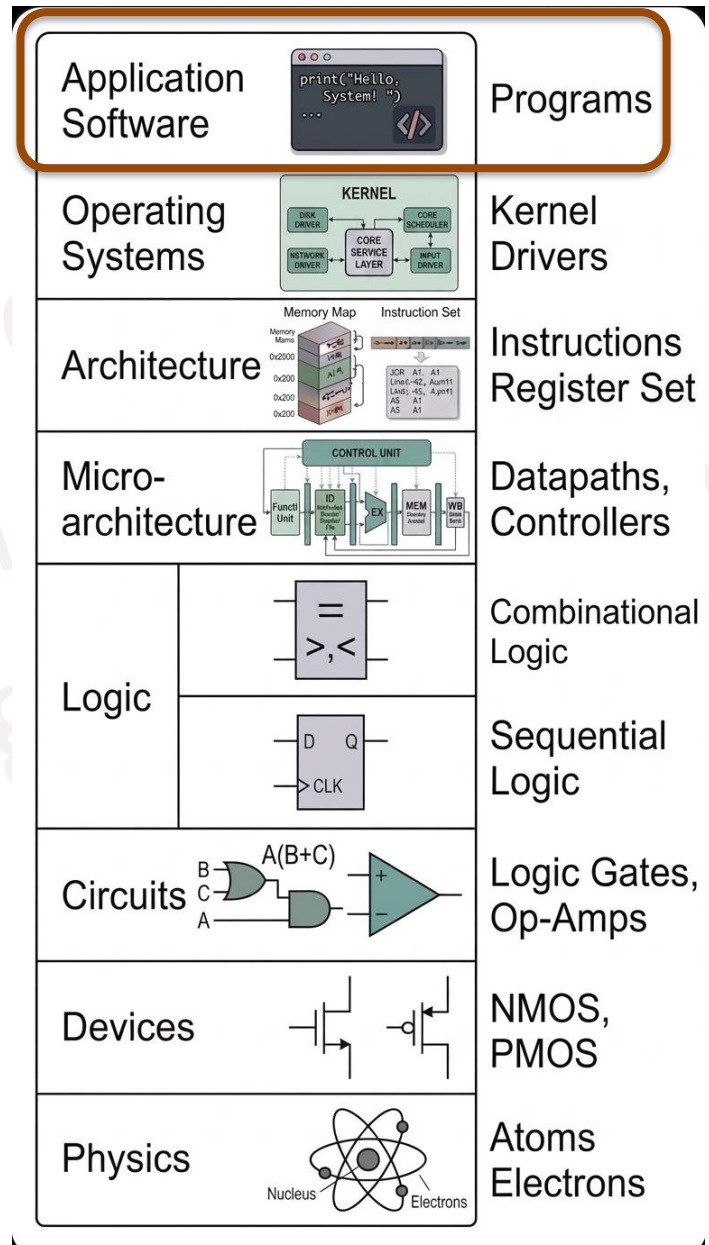
- **Space Multiplexing:**

Tài nguyên sẽ được chia thành nhiều phần nhỏ để nhiều chương trình sử dụng song song.



Tầng Application Software

1. Là **phần mềm** hoạt động thông qua các dịch vụ mà Operating Systems cung cấp.
2. Là các chương trình được thiết kế để thực hiện những chức năng phục vụ trực tiếp cho các nhu cầu của người dùng.
3. Ví dụ:
 - Trình duyệt web.
 - Phần mềm tính toán, soạn thảo văn bản,...
 - Các phần mềm khác được chính người dùng tạo nên trên nền phần mềm đã có.

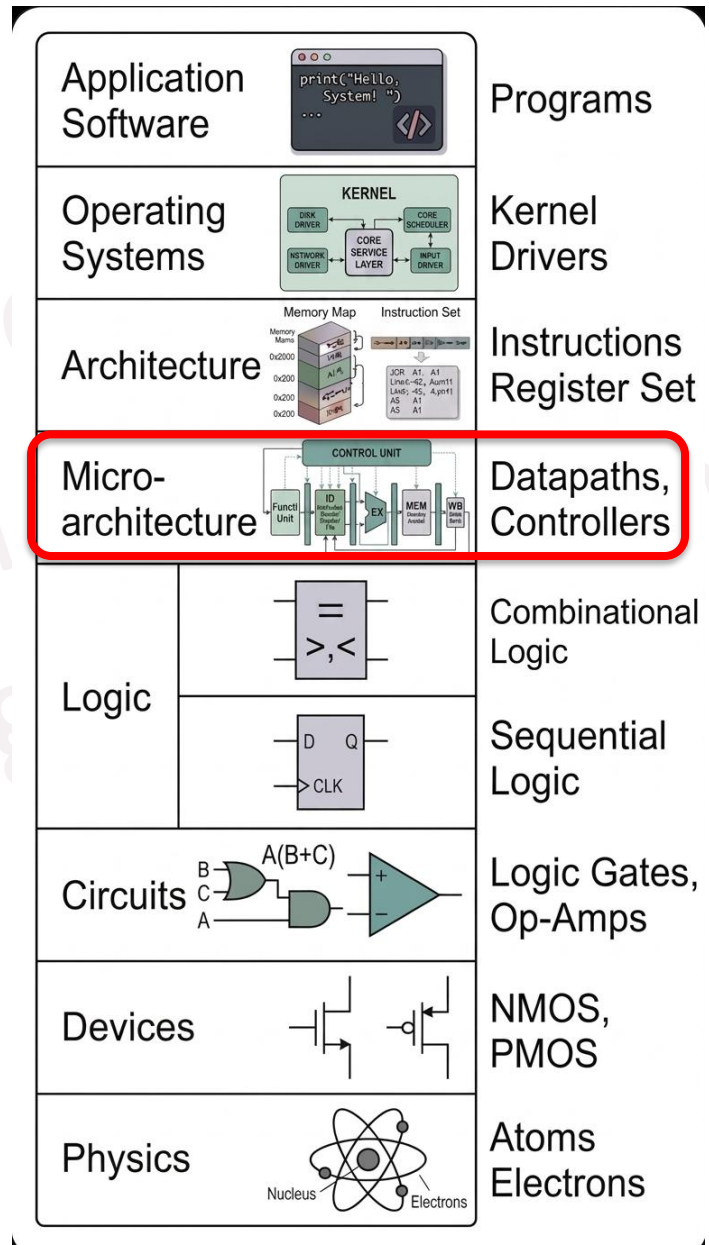


Tầng Microarchitecture

1. **Microarchitecture** mô tả cách **Architecture** được thực hiện bằng các phần cứng cụ thể.
2. Tầng này là **liên kết giữa tầng Logic và tầng Architecture**
3. Các thành phần ở tầng Logic được tập hợp để chuyển thành một cấu trúc phần cứng nhằm thực hiện một hoặc nhiều chức năng nhất định.

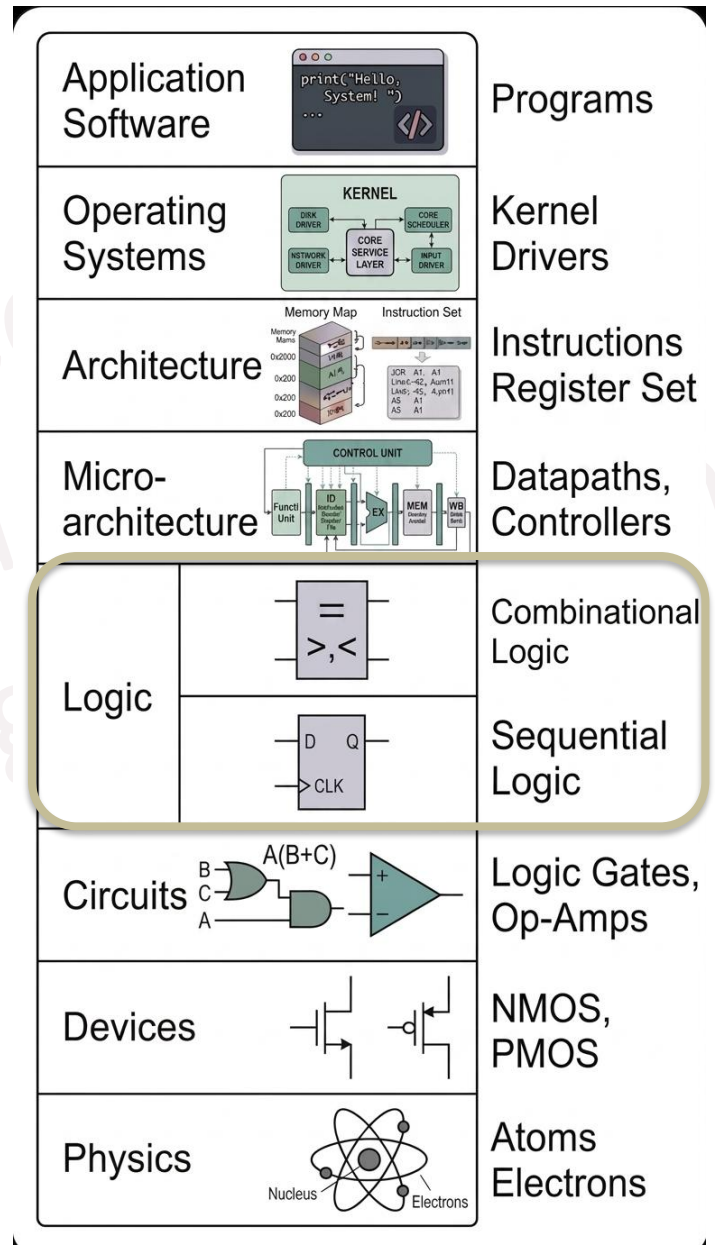
Lưu ý cần nhớ:

1. Một **Architecture** bất kỳ nào cũng có thể được thực hiện bằng **một hoặc nhiều Micro-architecture** (μ Arch).
2. Sự khác nhau giữa các μ Arch nằm ở **sự đánh đổi và kết hợp giữa các thành phần**:
 - Hiệu năng (Performance).
 - Công suất (Power).
 - Diện tích (Area).
 - Chi phí (Price).



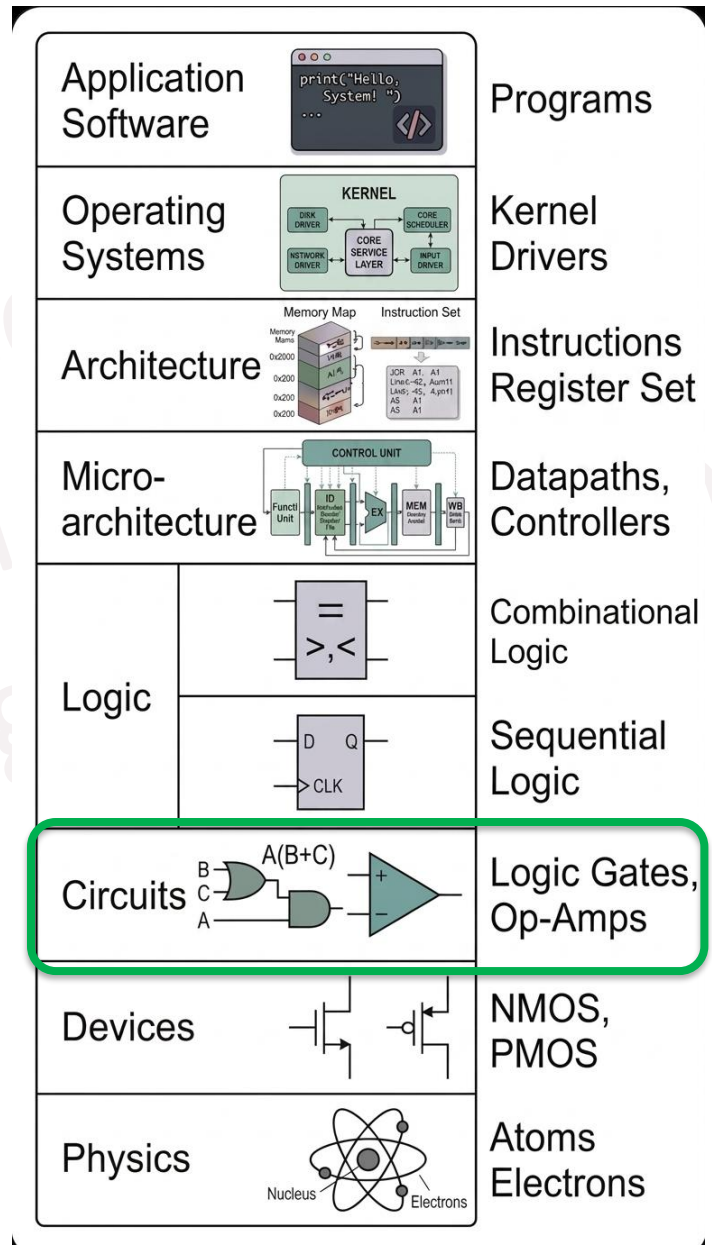
Tầng Logic

1. Đây là tầng thể hiện phần cứng của hệ thống bằng các phép toán và quan hệ logic.
2. Tầng Logic xác định hành vi tính toán còn μ Arch sẽ quyết định cách tổ chức và kết hợp các hành vi.
3. Có 2 loại logic nền tảng:
 - **Combinational Logic**: Kết quả dữ liệu ngõ ra chỉ phụ thuộc vào giá trị dữ liệu ngõ vào ở thời điểm hiện tại.
 - **Sequential Logic**: Kết quả dữ liệu ngõ ra phụ thuộc vào giá trị dữ liệu ngõ vào ở thời điểm hiện tại và trạng thái nội bộ đã được lưu từ các giá trị trong quá khứ.



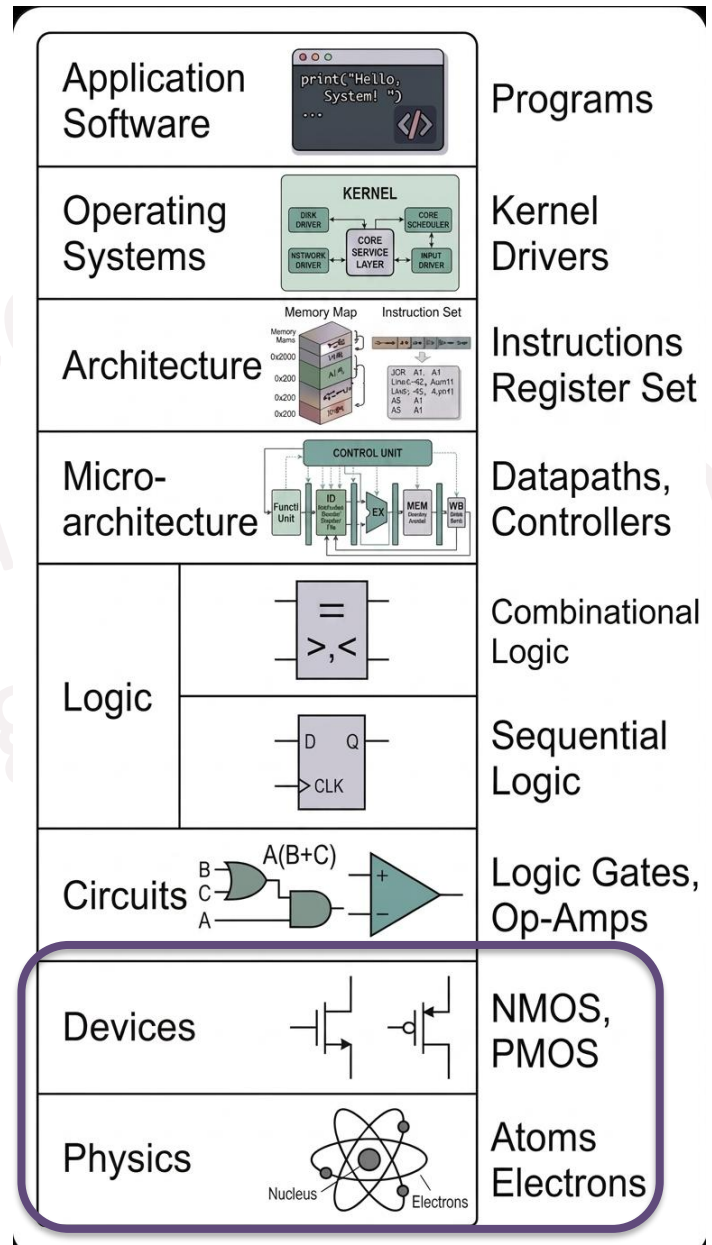
Tầng Circuits

1. Đây là tầng có nhiệm vụ hiện thực hóa các phép toán và quan hệ logic ở tầng trên bằng các **mạch điện cụ thể**.
2. Ở tầng này, các **logic** được chuyển **thành các tín hiệu điện có thể đo đạc và điều khiển**.
3. Circuits gồm 2 loại chính:
 - **Digital Circuits:**
 - Mạch xử lý điện áp ở các **khoảng giá trị rời rạc**.
 - Ví dụ: cổng logic, mạch cộng, mạch điều khiển,...
 - **Analog Circuits:**
 - Mạch xử lý điện áp ở các **khoảng giá trị liên tục**, cũng như **hỗ trợ một số hoạt động liên quan đến Digital Circuits**.
 - Ví dụ: mạch khuếch đại điện áp, mạch lọc nhiễu tín hiệu, mạch ổn định tín hiệu,...



Tầng Devices & Tầng Physics

1. Các mạch điện đề cập ở tầng trên được xây dựng từ **các linh kiện bán dẫn vật lý**, gọi chung là **Device**, chi tiết quan tâm ở **tầng Devices**.
2. Hoạt động của các Device lại bị chi phối bởi sự chuyển động của **các electron và lỗ trống** trong các vật liệu bán dẫn, chi tiết quan tâm ở **tầng Physics**.
3. Xét ở tầng Devices, phổ biến nhất là các MOSFET dạng tăng cường được điều khiển bằng điện áp (Enhancement-mode Metal-Oxide-Semiconductor Field-Effect-Transistor):
 - **NMOS**: gọi tắt của MOSFET kênh N (hạt tham gia hoạt động là **electron**).
 - **PMOS**: gọi tắt của MOSFET kênh P (hạt tham gia hoạt động là **lỗ trống**).



Tài Liệu Tham Khảo

1. Digital Design and Computer Architecture – Section 1.2
2. Introduction to Computing Systems – From bits & gates to C/C++ & beyond – Section 1.7
3. The MINIX book: Operating Systems – Design and Implementation – Section 1.1